new relic

# Ingestion Team

# AI-Led Development

# Environment

*Claude Code + GitHub Copilot + New Relic MCP*

🚀 Rapid Development

🔍 Instant Logs & RCA

⚡ Multi-Agent Orchestration

🧠 Compound Knowledge

📈 3× Productivity

# The Challenge — Four Real Pains the DataOS Ingestion Team Was Living With

### 🕐 Slow Development Tickets

**2–3 DAYS**

per ticket — regardless of complexity or volume

### 🔍 Pipeline Debugging

**30 MIN–3 HRS**

backtracking alert issues one by one manually

### 💥 Hidden Blast Radius

**5-DAY**

incident caused by a change no one knew would break

### 🗂️ No Historical Context

**HOURS–DAYS**

re-discovering repo structure and past decisions
every time someone joined, left, or came back

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# DataOS Ingestion — How We Run AI-Led Development Today

## What We Own

DATA SOURCES

Billing Platform · Kafka
SFDC · Zuora · etc.

↓

INGESTION — WE OWN THIS

**FiveTran · Kafka · NRDB**
**Spark · Custom Pipelines**

↓

DOWNSTREAM ZONE 2

Consumer · Data Lake
Dashboard

## Daily Workflow

1. **Start with full context**
   Memory + codebase graph auto-load in seconds
2. **Copilot: lines + blind spot check**
   Autocomplete in IDE · cross-model review catches what Claude misses
3. **Skills trigger automatically**
   /airflow · /pyspark · /nrql · /terraform · /sql
4. **Incidents diagnosed via NR MCP**
   30 min–3 hrs → 0–20 min for alert issues
5. **/avengers for big tasks**
   5 parallel agents — 2–3 days → 2–6 hours
6. **/wrap-up closes the loop**
   Jira updated · knowledge captured · graph refreshed

## Tools in Use

**Claude Code**
20 skills · memory · /avengers
Team environment AI

**GitHub Copilot**
Line completion · inline chat
Cross-model review (covers Claude blind spots)

**NR MCP**
Live logs · metrics · traces
Diagnose without leaving chat

**Jira + Confluence**
Tickets + docs auto-updated
on every /wrap-up

**Code Graph**
28 tools · blast radius
Impact analysis before coding

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Two AI Tools. Different Jobs. Better Together.

## GitHub Copilot — In the IDE

- ✅ Autocomplete lines & functions as you type
- ✅ Inline chat: explain, refactor, or debug code
- ✅ Generate tests and boilerplate instantly
- ✅ Cross-model review: GPT catches Claude blind spots

***Together: deliberate cross-model coverage —***

Different AI models catch different blind spots

Claude leads · Copilot verifies · humans approve

+

## Claude Code — Team Environment AI

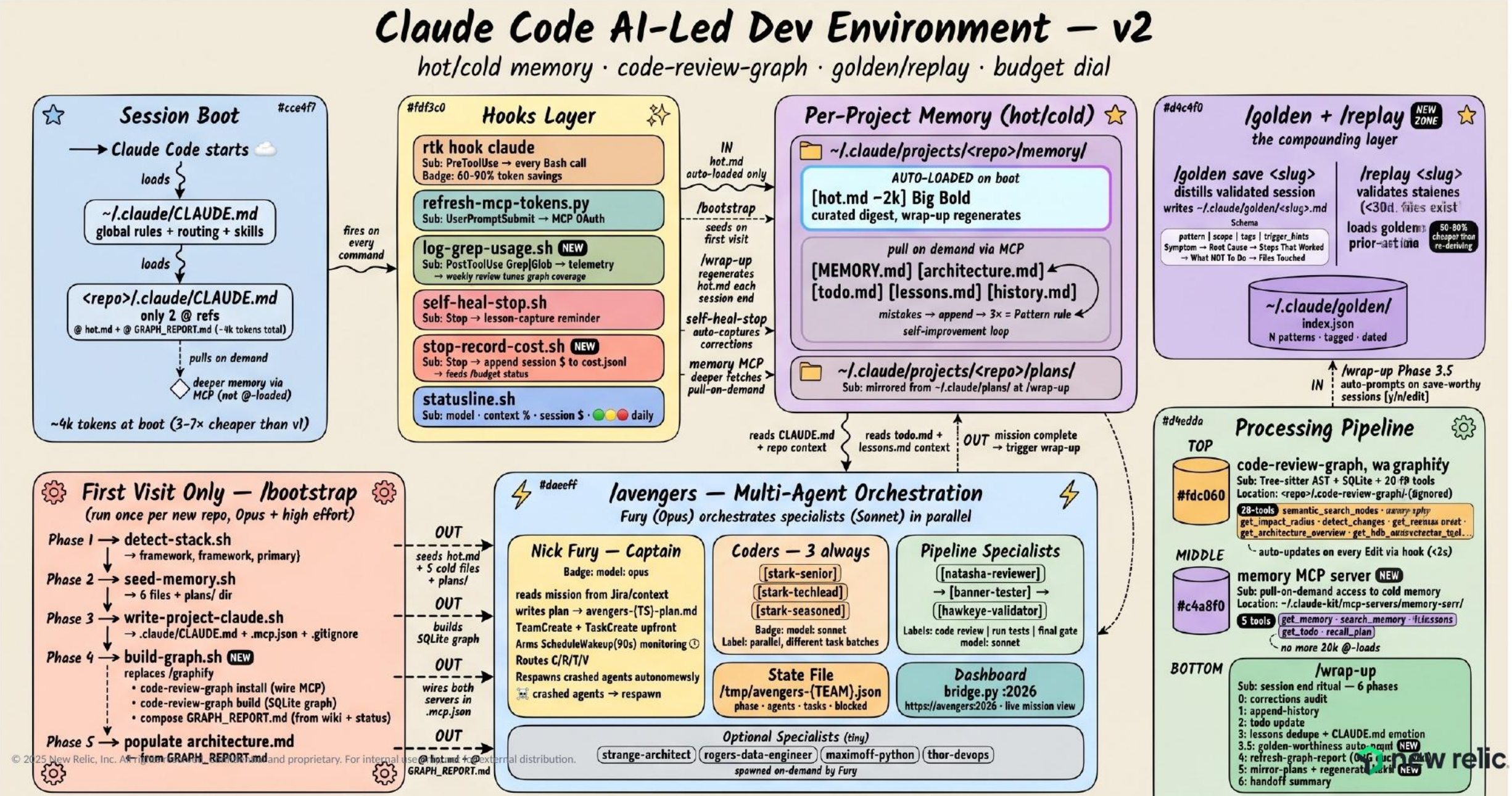
- ✅ Full codebase context at every session start
- ✅ Queries NR, Jira, Confluence live in the chat
- ✅ Blast radius visible before writing a single line
- ⚡ **/avengers: 5 parallel agents — 2–3 days → 2–6h**
- ⚡ **Incident diagnosis via NR MCP — 0–20 min**
- ✅ 20 skills auto-trigger on file type or topic
- ✅ Jira + Confluence auto-update on every session end
- ✅ 5-layer review: CC Opus → Copilot GPT (blind spots) → agents → human

**Together: 8–12× overall productivity for DataOS Ingestion DE tasks — reported by the team**

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# System Architecture — How It All Connects Under One Environment




© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only. Not for external distribution.

# Full AI-Led Development Lifecycle — Time Savings at Every Phase

| | 01 Requirement | 02 Spec & Plan | 03 Architect | 04 Build | 05 Review | 06 Test & Gate | 07 Deploy | 08 Monitor | 09 Document | 10 Learn |
|---|---|---|---|---|---|---|---|---|---|---|
| | Manager / PM | AI-assisted | AI + Graph | Task & volume dep. | 5-layer AI+Human | AI auto-mated | Human initiates | NR MCP (AI) | AI auto-update | Golden patterns |
| Before: | — | 4–6h | 2–4d | 2–3d | — | — | — | 1–10h | manual | — |
| After: | — | 20m–1h | 1–2d | 2–6h | — | — | — | 0–20m | auto | auto |

**Confirmed by engineer · Spec+Arch+Build: days → hours · Alert issue diagnosis: hours → 0–20 min · Overall: 8–12×**

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Faster Development — Days Became Hours

**BEFORE**

## 2–3 DAYS

Per development ticket — regardless of complexity
Context rebuilt from scratch every time

→

**AFTER**

## 2–6 HOURS

Task & volume dependent
Full context auto-loaded · skills auto-trigger · graph aware

### Spec & Plan
**4–6h → 20m–1h**

AI assists planning from existing codebase patterns — not from scratch

### Architecture
**2–4d → 1–2d**

Code graph shows impact + CDD & Confluence auto-updated

### Context
**always ready**

Memory + graph auto-load
No archaeology every session

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# /avengers — Five AI Engineers Working in Parallel on One Task

Tasks that take one engineer one full day, completed in under 2 hours:

**Orchestrator (Opus model — plans, routes, validates, shuts down)**

| Planner | Coder ×N | Code Reviewer | Validator |
|---|---|---|---|
| Researches codebase → implementation plan | Parallel code writing verifies build passes | Code standards + security checks | Tests + impact analysis before any merge |

1. Orchestrator reads spec → spawns Planner + Coders simultaneously
2. Each agent works on an independent file — zero waiting
3. Reviewer + Validator run in parallel on all output
4. Human sees a single clean PR to approve

**1 DAY**

↓

**2 HOURS**

Airflow DAG + dbt model
+ NR alert — all in parallel

**Engineer reviews**
**1 PR instead of 5 files**

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Incident Investigation — AI Queries New Relic Directly, Instantly

## BEFORE — Manual hunt

**2–3 HOURS**

Log in to NR UI · navigate to entity · scroll logs
switch tabs · compare charts · Slack teammates

Form hypothesis → test → repeat → still uncertain
Customer impact growing every minute

→

## AFTER — AI-powered diagnosis

**30 MINS**

"analyze kafka consumer lag last 4 hours"
Claude queries NR, returns structured analysis

Consumer lag · partition skew · throughput drop
Affected services identified — fix scoped immediately

**12 New Relic tools · Jira auto-updated · root cause in minutes**

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Impact Analysis — Know What Breaks Before Writing a Single Line

## We see the full impact of every change BEFORE it touches production.

### ⚠ Before: Invisible blast radius

Engineer changes a pipeline file.
Deploys. A downstream consumer breaks silently.
5-day incident — all from one undetected dependency.

→

### ✅ After: Instant clarity

Impact analysis runs before coding.
Sees 3 callers, 1 downstream consumer.
Fix scoped. Ships confidently. Zero incident.

**~1 sprint/qtr**

absorbed in surprise rework — before

**< 1 day**

rework after impact-aware development

**0 blind deploys**

every change impact-checked first

**Auto-updates**

dependency map rebuilt on every file save

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Documentation That Writes Itself — Jira & Confluence Always Up to Date

Work session completes → /wrap-up fires (~30s) → Claude drafts summary → Jira ticket auto-updated → Confluence published

## Jira — What gets auto-updated

| | |
|---|---|
| **Status** | In Review → Done · automatically |
| **Comment** | What was done + decisions made |
| **Worklog** | Time logged from session duration |
| **Links** | Commits linked via branch name |

## Confluence — What gets published

| | |
|---|---|
| **Architecture** | Decisions logged after every session |
| **Postmortem** | Incident page auto-created on wrap-up |
| **Release** | Notes from git history — auto-versioned |
| **Runbooks** | Updated as team learns new patterns |

**Zero manual effort. Every session documented. Every time.**

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Knowledge That Compounds

*Every session makes every future session smarter — knowledge never leaves the team*

ALWAYS LOADED

### Session memory — auto-loads at every session start

Active priorities + recent lessons + architecture summary loaded automatically. No manual setup. Zero tokens wasted.

PULL ON DEMAND

### Team knowledge — retrieved only when relevant

Patterns · architecture decisions · session history · past plans. Fetched on demand — only what's needed, when it's needed.

REPLAYABLE WINS

### Validated fixes — saved as reusable templates

After a confirmed fix: the steps are captured and indexed. Next time the same problem appears, the proven solution loads instantly.

**Every session: team knowledge grows. Every fix: the next one is faster. The system gets smarter, not staler.**

# The Results — Numbers the Team Has Measured

**8–12×**

overall productivity
for DE tasks

**2–6h**

dev ticket
vs 2–3 days before

**0–20m**

Alert issue diagnosis
vs 1–10 hours before

**25+ SP**

sprint capacity
+ adhoc on top

**More measurements**

**5–15 min**

context reset after absence
(vs hours/days before)

**3–5h**

new pipeline (full cycle)
vs 2–4 days before

**4–6h → 20m–1h**

spec & plan phase

**2–4d → 1–2d**

architecture + CDD
+ Confluence

**8–12× overall for DE tasks · All metrics reported directly by Ingestion Team engineers**

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Before & After — Ingestion Team · 3 Repos · 3 Months

| What We Measured | Before | After (AI-Led) |
|---|---|---|
| **Spec & Plan** | 4–6 hours | **20 min – 1 hour** ✅ |
| **Architecture + CDD** | 2–4 days | **1–2 days** ✅ |
| **Development ticket** | 2–3 days | **2–6 hours (task-dependent)** ✅ |
| **New pipeline (full cycle)** | 2–4 days | **3–5 hours** ✅ |
| **Alert issue diagnosis** | 30 min – 3 hours | **0–20 minutes (NR MCP)** ✅ |
| **Context after absence** | Hours / days | **5–15 min via /bootstrap** ✅ |
| **Sprint throughput** | 25 SP (at capacity) | **25 SP + additional + adhoc** ✅ |
| **Documentation effort** | Manual, often skipped | **Automatic via /wrap-up** ✅ |
| **Code review layers** | Manual review only | **CC Opus + Copilot + Natasha + Human** ✅ |

✅ = confirmed by Ingestion Team engineers · Repos: om-airflow-dags · spark-kafka-apps · tf-dataos-new-relic

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# AI Does the Work. Humans Stay in Control. Always.

| | | |
|---|---|---|
| 1 | **Requirements** | Human writes the spec — AI reads it, asks clarifying questions |
| 2 | **Architecture** | Human approves the implementation plan — before a line is written |
| 3 | **Security Review** | AI flags every security vulnerability — any critical issue blocks the pipeline |
| 4 | **PR Merge** | Human reviews every PR — agent output treated like any other code |
| 5 | **Deploy** | Human initiates the deploy — AI writes release notes + monitors |

*Agent PRs treated identically to human PRs — same review, same standards, same gates.*

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

# Getting Started — From Zero to Productive in 4 Steps

## 15 MIN

### Step 1 — Install & Setup

*One-time per engineer*

Claude Code + MCP servers + Budget dial
Multi-orchestration + hooks + 20 skills
Fully automated — one script.

## 5 MIN

### Step 2 — Bootstrap New Repo

*First visit to any repo*

Detects your stack, seeds team memory, builds codebase graph, wires all MCPs.
Runs once — context ready forever.

## ALWAYS

### Step 3 — AI-Led Development

*Single agent or multi agent*

Single task: Claude Code works alongside you.
Complex task: /avengers spawns parallel agents.
Copilot handles lines. Claude handles investigations.

## 30 SEC

### Step 4 — Wrap-Up

*Every session end*

Jira auto-updated · lessons captured
Knowledge persisted · graph refreshed.
The habit that makes everything compound.

## The Compounding Effect

### Day 1

Full codebase context ready.
Development tickets: days → hours.

### Month 1

Lessons accumulate. Patterns reused.
Alert diagnosis: hours → 0–20 min.

### Month 3+

Knowledge never leaves the team.
8–12× productivity — sustained.

© 2025 New Relic, Inc. All rights reserved. Confidential and proprietary. For internal use only, not for external distribution.

new relic

new relic

# What We're Asking

**Support adoption**

Drive AI-led workflows across all Data Engineering & Management in DTM.

**Allocate time**

Support the daily wrap-up habit — 30 seconds per session. Knowledge compounds automatically.

**Endorse the model**

Establish this as the standard template for all DE teams at New Relic.

**8–12×**

*Productivity*

**2–6h**

*Dev ticket*

**0–20m**

*Alert diagnosis*

**25+ SP**

*Sprint capacity*

## Thank You · Questions?

*Repo: github.com/nr-mdakilahmed/claude_code_setup*